# ORGANISASI & ARSITEKTUR KOMPUTER



T# topik → dipakai di soal Q# · F#x kode rumus, dirujuk di jawaban hal. 2 · ↻ VARIASI kalau soalnya diubah · † buku teks, bukan dari file kuis

## T1 REPRESENTASI DATA → Q1 Q2

### F1a Hex ↔ Biner (1 digit hex = 4 bit)

| | | | |
|---|---|---|---|
| 0 0000 | 1 0001 | 2 0010 | 3 0011 |
| 4 0100 | 5 0101 | 6 0110 | 7 0111 |
| 8 1000 | 9 1001 | A 1010 | B 1011 |
| C 1100 | D 1101 | E 1110 | F 1111 |

1 byte = 2 hex · 16 b = 4 hex · 24 b = 6 · 32 b = 8. **Nol di depan tetap ditulis** (4→0100).

### F1b Byte dalam word 32-bit

```
0x 77 88 99 AA = B3 B2 B1 B0
tengah = (x >> 8) & 0xFFFF
```

### F1c Two's complement — tanda

- **MSB = bit tanda**: 0 → positif, 1 → negatif · digit hex pertama **0–7 → +, 8–F → −**
- Rentang †: $-2^{n-1}$ … $2^{n-1}-1$ (16-bit: −32,768 … 32,767)
- Nilai negatif = unsigned − $2^n$ · negasi †: balik bit, +1

### F1d Karakter packed (2 per word 16-bit)

Karakter ke-1 = high byte: "ME" → `4D|45` = `0x4D45`. ASCII 7-bit (`00–7F`) † → MSB 0 → selalu positif.

**↻ VARIASI SOAL — kalau ditanya…**
- **Ambil byte atas/bawah?** atas = `x >> 24`, bawah = `x & 0xFF`
- **Nilai desimal signed?** MSB 1 → unsigned − $2^{16}$: 0x8899 = 34,969 − 65,536 = −30,567
- **String/huruf lain?** ASCII huruf-1 ‖ huruf-2, lalu cek MSB (F1c)

## T2 HUKUM MOORE → Q3

Performa ×2 setiap **2 tahun** (asumsi kuis).

**F2a** Faktor = $2^{(\Delta\text{tahun} / 2)}$
**F2b** CPU lebih lama: T = $T_{kini}$ × faktor
**F2c** CPU lebih baru: T = $T_{kini}$ ÷ faktor

**↻ VARIASI SOAL — kalau ditanya…**
- **CPU 4 thn lebih baru?** 200 ÷ 4 = 50 s (F2c)
- **Berapa tahun agar X× lebih cepat?** Δ = $2 \cdot \log_2 X$ (8× → 6 thn)

## T5 ASSEMBLER → Q5

Label + koma: `Start,` · `DEC n` konst. desimal · `HEX n` konst. hex · `ORG` alamat awal † · 2 pass †.

**F5a** Tabel simbol: label → alamat hex
**F5b** Kode hex = opcode ‖ alamat 3 digit
`LOAD P (P=106) → 1106 · OUTPUT → 6000`

alamat tiap baris (+1) → tabel simbol → mnemonik→opcode, label→alamat → tanpa operand = `000`

**↻ VARIASI SOAL — kalau ditanya…**
- **ORG beda (mis. ORG 200)?** semua alamat & tabel simbol ikut geser (LOAD P → 1206)
- **Data dalam hex?** DEC 60/40/15 = 003C/0028/000F, S akhir = 0055
- **Ada Jump/Skipcond (loop)?** trace per baris, catat PC & AC tiap langkah (T3)

## T3 ARSITEKTUR MARIE → Q4 Q5

Word & instr 16-bit · 4K word, word-addressable · alamat 12-bit †

### F3a Format instruksi

bit 15 … bit 0

| OPCODE 15–12 (4 b) | ADDRESS 11–0 · 12 bit (3 digit hex) |
|---|---|

`4155` → `op 4 (Subt) | alamat 155h`

### Register † lebar

| Reg | Bit | Fungsi |
|---|---|---|
| AC | 16 | akumulator (operand/hasil) |
| MAR | 12 | alamat memori yang diakses |
| MBR | 16 | data ke / dari memori |
| PC | 12 | alamat instr *berikutnya* |
| IR | 16 | instruksi saat ini |

### F3b Set instruksi (★ dipakai di kuis)

| Hex | Instr | Aksi |
|---|---|---|
| 1 | Load X ★ | AC ← M[X] |
| 2 | Store X ★ | M[X] ← AC |
| 3 | Add X ★ | AC ← AC + M[X] |
| 4 | Subt X ★ | AC ← AC − M[X] |
| 6 | Output ★ | AC → output |
| 7 | Halt ★ | hentikan program |

† lainnya: **0** JnS X (M[X]←PC, PC←X+1) · **5** Input (AC←input) · **8** Skipcond (lompati instr berikut jika 000: AC<0, 400: AC=0, 800: AC>0) · **9** Jump X (PC←X) · **A** Clear (AC←0) · **B** AddI X (AC←AC+M[M[X]]) · **C** JumpI X (PC←M[X]) · **D/E** LoadI/StoreI (tidak langsung)

## T4 SIKLUS INSTRUKSI → Q4

`FETCH → DECODE → operand → EXECUTE ↺`

**F4a** Fetch: MAR ← PC · MBR ← M[MAR]
IR ← MBR · PC ← PC + 1
**F4b** Decode: opcode = IR[15–12], alamat = IR[11–0]

| F4c | Execute (RTL) |
|---|---|
| Subt X | MAR←IR[11–0] · MBR←M[MAR] · AC←AC−MBR |
| Load X † | MAR←X · MBR←M[MAR] · AC←MBR |
| Add X † | seperti Subt, tapi AC←AC+MBR |
| Store X † | MAR←X · MBR←AC · M[MAR]←MBR |

**↻ VARIASI SOAL — kalau ditanya…**
- **Opcode lain?** 1xxx → AC = M[xxx] · 3xxx → AC + M[xxx] · 2xxx → M[xxx] = AC (AC tetap) · 9xxx → PC = xxx
- **Nilai register akhir?** ikuti RTL. Q4: IR = 4155, MAR = 155, MBR = 0010, AC = 0025, PC = 151
- **PC akhir?** PC+1 sudah terjadi saat fetch, kecuali Jump/Skipcond mengubahnya

## T6 EXPANDING OPCODE → Q6

**Bit operand banyak → opcode pendek.** Pola opcode sisa jadi **escape prefix** untuk instr dengan bit operand lebih sedikit.

**F6a** Pola terpakai = n × $2^b$ (b = total bit operand)
**F6b** Bisa ⇔ $\Sigma\, n \cdot 2^b \le 2^W$ (W = 16 → 65,536)

**↻ VARIASI SOAL — kalau ditanya…**
- **Maks. instr tipe terakhir?** sisa = $2^W$ − Σ tipe lain; maks = ⌊sisa ÷ $2^b$⌋. Q6: (65,536 − 51,200) ÷ 256 = 56
- **Tidak muat?** Σ > $2^W$ → tidak bisa, tulis selisihnya
- **Format bukan 16-bit?** ganti $2^W$ (mis. 12-bit → 4,096)

## T7 PIPELINING → Q7

k segmen · τ clock · n instr · $t_n$ = 1 instr tanpa pipeline

**F7a** $t_n = k \times \tau$ (ideal)
**F7b** $T_{non\text{-}pipe} = n \times t_n$
**F7c** $T_{pipe} = (k + n - 1) \times \tau$
**F7d** $S = T_{non\text{-}pipe} / T_{pipe}$ ; n → ∞ ⇒ S → k

```
k=5 siklus 1  2  3  4  5  6
Seg 1      I1 I2 I3 I4 I5 I6
Seg 2         I1 I2 I3 I4 I5
Seg 3            I1 I2 I3 I4
Seg 4               I1 I2 I3
Seg 5                  I1 I2
```

Instr ke-1 selesai setelah k siklus, lalu 1 per siklus.

**↻ VARIASI SOAL — kalau ditanya…**
- **Speedup maksimum?** S → k = 5
- **Hanya waktu pipeline?** F7c saja (Q7: 32,160 ns)
- **$t_n$ diberikan ≠ k·τ?** pakai $t_n$ yang diberikan di F7b

## T8 CACHE DIRECT-MAPPED → Q8

**F8a** # blok MM = ukuran MM ÷ ukuran blok
**F8b** Bit alamat = $\log_2$(byte MM)
**F8c** Offset = $\log_2$(ukuran blok)
**F8d** Index = $\log_2$(# blok cache)
**F8e** Tag = alamat − index − offset

| TAG 10 | INDEX 10 | OFFSET 4 |
|---|---|---|

**F8f** hex → biner (lebar penuh) → pecah **dari kanan** → index → desimal = blok cache

**↻ VARIASI SOAL — kalau ditanya…**
- **Tag berapa?** bit tag → desimal: 0101101000 = 360
- **Nomor blok MM?** alamat >> offset: 0x5A3C8 = 369,608 · **offset?** F = 15
- **Dua alamat bentrok?** index sama, tag beda → saling menimpa
- **Ukuran cache?** blok × ukuran blok = 1024 × 16 = 16 KB
- **Set-associative?** † # set = blok ÷ way; index = $\log_2$(# set)

# KUNCI JAWABAN KUIS & PENYELESAIAN



**Q1** T1 2 byte tengah — **0x8899** = `1000 1000 1001 1001`

**Q2** T1 tanda 0x4D45 — `0100 1101 0100 0101` → **POSITIF**

**Q3** T2 Moore, 4 thn lebih lama — 200 s × 4 = **800 s**

**Q4** T3 T4 MARIE 4155 — Subt 155 · AC = **0025** (hex)

**Q5** T3 T5 program MARIE — 1106 3107 4108 2109 6000 7000 · AC = **85**

**Q6** T6 expanding opcodes — 64,000 ≤ 65,536 → **BISA**

**Q7** T7 speedup pipeline — 160,000 / 32,160 ≈ **4.975**

**Q8** T8 cache direct-mapped — $2^{20}$ blok · 10|10|4 · **blok 968**

## Q1 Kompresi 32-bit → 16-bit — T1 F1b F1a

Ambil 2 byte tengah dari `0x778899AA`; tulis hex + biner.

- F1b Buang MSB `77` dan LSB `AA` → byte tengah `88`, `99`.
- 16-bit hex = `8899`.
- F1a 8=1000, 8=1000, 9=1001, 9=1001.

▶ **JAWABAN: 0x8899 = 1000 1000 1001 1001**

⚡ **TIPS UJIAN** Tidak ditanya: MSB = 1 → jika dibaca signed, nilainya negatif.

## Q2 "ME" dibaca integer — T1 F1d F1a F1c

`0x4D45` ('M'=4D, 'E'=45). Biner? Tanda two's complement?

- F1a 4=0100, D=1101, 4=0100, 5=0101.
- Biner: `0100 1101 0100 0101`
- F1c MSB (bit paling kiri) = **0** → positif dalam two's complement.

▶ **JAWABAN: 0100 1101 0100 0101 → POSITIF (MSB = 0)**

## Q3 Hukum Moore — T2 F2a F2b

200 s di CPU sekarang; CPU 4 tahun **lebih lama**; performa ×2 tiap 2 thn.

- 4 thn ÷ 2 = 2 periode penggandaan.
- F2a CPU lama $2^2$ = 4× lebih lambat.
- F2b 200 s × 4.

▶ **JAWABAN: 800 detik**

## Q4 Siklus MARIE — T3·T4 F3a F4a F4c

PC = 150, M[150] = `4155`, M[155] = `0010`, AC = `0035`.

F3a **1. Decode:** opcode **4** = **Subt**; alamat operand **155** (hex).

F4a **2. Fetch:** `MAR←PC · MBR←M[MAR] · IR←MBR · PC←PC+1`

F4c **3. Execute (Subt 155):**
`MAR←IR[11–0] · MBR←M[MAR] · AC←AC–MBR`
AC = `0035` − `0010`

▶ **JAWABAN: AC akhir = 0025 (hex)**

Dalam desimal: 53 − 16 = 37.

## Q5 Assemble & trace — T3·T5 F5a F5b F3b

100: LOAD P, ADD Q, SUBT R, STORE S, OUTPUT, HALT · 106: P DEC 60 · Q DEC 40 · R DEC 15 · S HEX 0000

F5b **i. Kode hex** = opcode + alamat 3 digit

| Alamat | Instr | Op | Operand | Hex |
|---|---|---|---|---|
| 100 | LOAD P | 1 | 106 | **1106** |
| 101 | ADD Q | 3 | 107 | **3107** |
| 102 | SUBT R | 4 | 108 | **4108** |
| 103 | STORE S | 2 | 109 | **2109** |
| 104 | OUTPUT | 6 | 0 | **6000** |
| 105 | HALT | 7 | 0 | **7000** |

F5a **ii. Tabel simbol** (label → alamat hex)

| Label | Hex Address |
|---|---|
| **Start** | 100 |
| **P** | 106 |
| **Q** | 107 |
| **R** | 108 |
| **S** | 109 |

Alamat = baris tempat label ditulis. **Start** di baris instruksi pertama (100). **P, Q, R, S** = data setelah HALT (105), jadi mulai 106.

F3b **iii. Trace** (aksi tiap instr)

| Alamat | Instr | Memori | AC (des) |
|---|---|---|---|
| Awal | P, Q, R = 60, 40, 15 | | 0 |
| 100 | LOAD P | | 60 |
| 101 | ADD Q | | 60+40 = 100 |
| 102 | SUBT R | | 100−15 = 85 |
| 103 | STORE S | M[109]=85 | 85 |
| 104 | OUTPUT | output 85 | 85 |
| 105 | HALT | berhenti | 85 |

▶ **JAWABAN: AC = 85 (desimal) = 0x0055**

## Q6 Expanding opcode — T6 F6a F6b

5 instr × tiga opnd 4-bit · 30 × dua 5-bit · 50 × satu 8-bit

| Jenis | F6a Pola bit dibutuhkan |
|---|---|
| 5 × (3×4) | 5 × $2^{12}$ = 20,480 |
| 30 × (2×5) | 30 × $2^{10}$ = 30,720 |
| 50 × (1×8) | 50 × $2^8$ = 12,800 |
| F6b **Total** | **64,000 ≤ $2^{16}$ = 65,536** |

▶ **JAWABAN: BISA — sisa 1,536 pola**

**Contoh susunan opcode** (cek sendiri): `0000`–`0100` (5 instr) · `010100`–`110001` (30) · `11001000`–`11111001` (50) · sisa `11111010`–`11111111`

## Q7 Speedup pipeline — T7 F7a–d

k = 5, τ = 40 ns, n = 800 · F7a $t_n = k \cdot \tau$ = 200 ns

F7b $T_{non\text{-}pipe} = n \cdot t_n$ = 800 × 200 = **160,000 ns**

F7c $T_{pipe} = (k+n-1) \cdot \tau$ = (5+800−1) × 40 = 804 × 40 = **32,160 ns**

F7d S = 160,000 / 32,160

▶ **JAWABAN: S ≈ 4.975**

⚡ **TIPS UJIAN** Cek logika: untuk n besar, S harus sedikit di bawah k = 5.

## Q8 Cache direct-mapped — T8 F8a–f

MM $2^{24}$ B byte-addressable · 1024 blok cache · 16 B / blok

F8a **1. Blok MM** = $2^{24}$ / $2^4$ = **$2^{20}$ blok**

**2. Format alamat**

- F8b Total = $\log_2 2^{24}$ = 24 bit
- F8c F8d Offset = $\log_2 16$ = 4 · Index = $\log_2 1024$ = 10
- F8e Tag = 24 − 10 − 4 = 10

▶ **JAWABAN: Tag 10 | Index 10 | Offset 4**

F8f **3. Petakan 0x5A3C8F** — pecah kanan → kiri

```
hex:  5    A    3    C    8    F
bin: 0101 1010 0011 1100 1000 1111

  0101101000 | 1111001000 | 1111
    TAG 10   |  INDEX 10  | OFF 4
```

Index $1111001000_2$ = 512+256+128+64+8

▶ **JAWABAN: Blok cache (index) 968**

⚡ **TIPS UJIAN** Trik hex: offset = 4 bit = digit hex terakhir → buang: `5A3C8`; index = 10 bit terbawah = `0x3C8` = 3·256 + 12·16 + 8 = **968**.

## REFERENSI CEPAT

| | | |
|---|---|---|
| $2^4$ = 16 | $2^8$ = 256 | $2^{10}$ = 1,024 |
| $2^{12}$ = 4,096 | $2^{16}$ = 65,536 | $2^{20}$ = 1M |
| $2^{24}$ = 16,777,216 (16M) | | |

| bit | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|
| bobot | 512 | 256 | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |

## KESALAHAN UMUM & TRIK

- **Nol di depan:** lengkapi dulu sampai lebar penuh (alamat 24-bit = 6 digit hex, 16-bit = 4).
- **Memecah alamat cache:** mulai dari **kanan** (offset dulu), jangan dari kiri.
- **Pipeline:** jangan lupa **−1** di (k+n−1); $t_n = k \cdot \tau$, bukan τ.
- **Expanding opcode:** jumlah × $2^b$, b = *semua* bit operand instr itu (3×4 = 12).
- **Alamat MARIE dalam hex:** `4155` → alamat `155` h; label diganti alamat hex.
- **DEC vs HEX:** `DEC 60` menyimpan 60 desimal = `003C` h.
- **Instr tanpa operand** (Output, Halt) → `X000`.
- **PC+1 terjadi saat fetch**, sebelum execute.
- **Cek tanda:** digit hex pertama ≥ 8 → negatif.
- **Moore:** CPU lebih lama → waktu × $2^{\Delta/2}$.